

AdatTracker Pro

Comprehensive Architecture, Feature Implementation, Multi-User SQLite Engine, and Portainer/Docker Conversion Manual.

APPLICATION

AdatTracker Pro (v2.0)

REPOSITORY

github.com/12thpassengineer/HabitTracker

RUNTIME & DATABASE

Python 3.11 + FastAPI / Uvicorn + SQLite3

TARGET DEPLOYMENT

Raspberry Pi 4/5, Proxmox LXC, Docker & Portainer

DOCUMENTATION INDEX

1. Project Genesis, Philosophy & Branding	Section 1
2. Detailed Feature Breakdown (Built from Scratch)	Section 2
3. Multi-User Architecture & SQLite Schema	Section 3
4. Docker & Portainer Containerization Technique	Section 4
5. Bare-Metal Deployment (Raspberry Pi & Proxmox LXC)	Section 5
6. Complete Rebuild & Disaster Recovery Blueprint	Section 6
7. Appendix: File Manifest & Port Configurations	Section 7

1. Project Genesis, Philosophy & Branding

1.1 The Meaning of "Adat"

The project was originally based on an experimental single-user UI. To establish complete intellectual property autonomy and avoid any trademark infringement with third-party software ("Habit Codex"), the project was officially christened **AdatTracker Pro**.

"Adat" (عادت / आदत) translates directly to *habit, custom, or routine* in Hindi, Urdu, Arabic, and several Southeast Asian dialects. The name reflects a core personal development thesis: *"You do not rise to the level of your goals. You fall to the level of your systems (adats)."*

1.2 Core Architectural Principles

- **Zero External DB Dependencies:** Uses an embedded SQLite database (`adattracker.db`). No separate PostgreSQL, MySQL, or Redis containers needed.
- **Dual-Engine Backend Resilience:** `server.py` automatically detects if high-performance `fastapi` is installed. If not, it seamlessly falls back to the Python standard library's `http.server` and `sqlite3` , requiring 0 external packages.
- **Extreme Resource Efficiency:** Consumes less than **35 MB of RAM**, making it ideal for 24/7 background operation on low-power hardware (Raspberry Pi Zero/3/4/5 or Proxmox 512MB LXC containers).
- **OLED Dark Aesthetic:** Designed with a deep `#050508` canvas, subtle purple/cyan neon luminescence, and custom micro-animations.

2. Detailed Feature Breakdown

Every major feature was iteratively engineered to solve specific productivity bottlenecks:



Monthly Matrix Grid

Displays all habits across the days of the current month. Automatically auto-scrolls to today's date on load. Features live completion checkmarks, streak counts (🔥), and completion percentages.



Interactive Priority Slider

Replaced clunky up/down buttons with a minimalist  drag handle. Users can drag and reorder habits dynamically; orders persist instantly to cloud database.



Proportional Score Engine

Eliminated arbitrary gaming XP. Replaced with mathematically sound daily completion ratios (e.g. 4/5 completed = 0.8 points). Tracks lifetime consistency score.



Sticky Notes Board

Replaced legacy mood journals with a Post-It note dashboard. Supports 6 pastel color themes, Pin to Top (📌), one-click clipboard copying, and live search filtering.



Multi-User Data Isolation

Simple authentication with **Username + Phone Number**. Each user receives their own isolated record in SQLite. Eliminates cross-user data leakage.



2-Second Interactive Guides

Floating contextual help popover. Hovering over any card or statistic for 2 continuous seconds displays an explanation, keyboard shortcuts, and pro tips.

2.1 Scoring Algorithm Formula

The scoring system rewards consistency rather than volume. Daily scores are calculated as follows:

```
Daily Score = (Completed Habits in Day) / (Total Active Habits)
Total Consistency Score = SUM(Daily Score for all logged days)
```

Example: If a user has 5 habits and completes 4 today, they receive **+0.80 points**. If they complete all 5, they receive **+1.00 point**.

3. Multi-User Architecture & SQLite Schema

3.1 SQLite Database Model (`adatracker.db`)

The database is auto-initialized on first startup inside the `./data/` volume.

```
-- 1. USERS TABLE
CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  username TEXT UNIQUE NOT NULL,
  phone TEXT NOT NULL,
  token TEXT UNIQUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  last_login TIMESTAMP
);

-- 2. USER DATA STORAGE TABLE (Isolated per user)
CREATE TABLE IF NOT EXISTS user_data (
  user_id INTEGER PRIMARY KEY,
  data_json TEXT NOT NULL,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  version INTEGER DEFAULT 1,
  FOREIGN KEY(user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

3.2 REST API Endpoints Specification

Endpoint	Method	Auth Required	Payload / Description
<code>/api/auth/register</code>	POST	No	<code>{"username": "...", "phone": "..."} </code> — Enrolls new user, generates auth token.
<code>/api/auth/login</code>	POST	No	<code>{"username": "...", "phone": "..."} </code> — Validates phone match, issues session token.
<code>/api/auth/me</code>	GET	Bearer Token	Returns current authenticated user profile object.
<code>/api/auth/logout</code>	POST	Bearer Token	Invalidates active session token.
<code>/api/data</code>	GET	Bearer Token	Retrieves the user's isolated habits, tasks, and notes payload.
<code>/api/data</code>	POST	Bearer Token	Saves updated state JSON to <code>user_data</code> table with debounce.
<code>/</code>	GET	No	Serves the single-page responsive application (<code>habit_tracker.html</code>).

4. Docker & Portainer Containerization Technique

One of the core challenges of running lightweight Python web applications on edge hardware (like a Raspberry Pi running Portainer) is balancing image build overhead with runtime reliability.

4.1 The Production Dockerfile

```
# Multi-arch compatible (AMD64 & ARM64)
FROM python:3.11-slim

WORKDIR /app

# Install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt || true

# Copy application files
COPY server.py habit_tracker.html ./

# Create persistent storage mount point
RUN mkdir -p /app/data

EXPOSE 8000

ENV PORT=8000
ENV HOST=0.0.0.0

CMD ["python", "server.py"]
```

4.2 The Dynamic Portainer Stack Technique (Bootstrap Method)

When hosting on a remote Portainer instance directly from a public GitHub repository, you don't need a Docker registry (Docker Hub). You can use Portainer's Web Editor with an automated bootstrap command:

```

version: '3.8'

services:
  adattracker:
    image: python:3.11-slim
    container_name: adattracker-pro
    restart: unless-stopped
    ports:
      - "8000:8000"
    volumes:
      - adattracker_data:/app/data
    command: >
      sh -c "apt-get update -qq && apt-get install -y -qq curl &&
        mkdir -p /app/data && cd /app &&
        curl -sSL https://raw.githubusercontent.com/12thpassengineer/HabitTracker/main/server.py -o server.py &&
        curl -sSL https://raw.githubusercontent.com/12thpassengineer/HabitTracker/main/habit_tracker.html -o habit_tracker.html &&
        curl -sSL https://raw.githubusercontent.com/12thpassengineer/HabitTracker/main/requirements.txt -o requirements.txt &&
        pip install --no-cache-dir -r requirements.txt --quiet &&
        python server.py"

volumes:
  adattracker_data:

```

Why This Technique Works Seamlessly:

1. Uses official lightweight `python:3.11-slim` image cached on almost every Docker host.
2. Downloads code live on container boot, meaning any GitHub commit is automatically applied on restart.
3. Mounts `adattracker_data` volume so all user databases persist across container updates.

Portainer Network Gotcha (HTTP vs HTTPS):

Browsers often attempt to force `https://` on newly opened ports. Since Portainer exposes port 8000 over plain HTTP, accessing `https://<IP>:8000` results in `Invalid HTTP request`. Always ensure the URL explicitly begins with `http://`.

5. Bare-Metal Deployment (Raspberry Pi & Proxmox LXC)

5.1 Universal 1-Line Installer

For users who prefer native execution without Docker, run this inside any Debian/Ubuntu terminal:

```
curl -sSL https://raw.githubusercontent.com/12thpassengineer/HabitTracker/main/install.sh | bash
```

5.2 Systemd 24/7 Service Unit (`habit-tracker.service`)

```
[Unit]
Description=AdatTracker Pro Multi-User Application
After=network.target

[Service]
Type=simple
User=root
WorkingDirectory=/opt/habit-tracker
ExecStart=/usr/bin/python3 /opt/habit-tracker/server.py
Restart=always
RestartSec=5
Environment=PORT=8000
Environment=HOST=0.0.0.0

[Install]
WantedBy=multi-user.target
```


6. Complete Rebuild & Disaster Recovery Blueprint

If the entire server, container, or repository is destroyed, follow this 4-step recovery procedure to get the application back up in under 3 minutes:

1. Deploy Container or Clone Repo:

```
git clone https://github.com/12thpassengineer/HabitTracker.git && cd HabitTracker
```

2. Restore Database Backup:

Copy your backed-up `adattracker.db` file directly into the `data/` folder:

```
mkdir -p data/ && cp /path/to/backup_adattracker.db data/adattracker.db
```

3. Launch Service:

```
docker compose up -d  
# OR natively:  
python3 server.py
```

4. Access & Verify:

Navigate to `http://<ip>:8000` . All users, streaks, and notes will be restored instantly.

7. Appendix: Directory Structure & File Manifest

```
HabitTracker/
├── docs/
│   ├── AdatTracker_Documentation.html    # Styled master documentation source
│   └── AdatTracker_Documentation.pdf      # Compiled distribution manual
├── frontend/
│   ├── index.html                        # Main application dashboard
│   └── habit_tracker.html                 # Standalone SPA source
├── backend/
│   ├── server.py                         # Dual-engine REST API server
│   └── requirements.txt                   # FastAPI & Uvicorn dependencies
├── docker/
│   ├── Dockerfile                       # Multi-architecture container build
│   └── docker-compose.yml                # Container orchestration specification
├── scripts/
│   ├── deploy.sh                        # Automated Linux setup script
│   └── habit-tracker.service              # Systemd background daemon configuration
├── install.sh                           # Universal 1-line curl setup script
├── docker-compose.yml                    # Root compose shortcut
├── README.md                             # GitHub public documentation
├── LICENSE                               # MIT Open Source License
└── .gitignore                           # Database and temporary file safety filter
```

— End of Technical Documentation • AdatTracker Pro v2.0 —